Integrating a Local Large Language Model (LLM) into a Raspberry Pi NAS

An extension to the project report "NAS Network Attached Storage & Video Streaming Using Plex"

Submitted by: Yuvraj Singh Sankhala | Supervisor: Mr. Surendra Choudhary Department of Computer Science & Engineering, Engineering College, Bikaner

> **Note for the author:** Items in [square brackets] are placeholders for your own measured values, screenshots, or choices. Performance figures in this draft are expected ranges, not measurements. Replace them with results from your own Raspberry Pi before submitting.

Abstract

The base project built a self-hosted Network Attached Storage (NAS) system on a Raspberry Pi 4 using OpenMediaVault (OMV) for storage management and Plex Media Server for streaming. This extension adds a locally hosted Large Language Model (LLM) to the same device, so that the NAS also acts as a private AI assistant. Using the Ollama runtime and a 4-bit quantized small language model, users on the local network can ask questions, summarize text, and draft content without sending any data to a third-party cloud service.

The integration keeps the original goals of the project: low cost, full data control, and privacy. The report describes the hardware limits of the Raspberry Pi 4, the model selection, installation, storage configuration on the external drive, network exposure, resource sharing with Plex, and testing. Results show that a 1B-3B parameter model is usable for short tasks on the Pi, with the expected trade-off of low speed and limited answer quality compared with cloud models.

Chapter 1: Introduction

1.1 Problem Definition

Cloud AI assistants are convenient, but every prompt leaves the user's network. Documents, notes, and personal data are processed on servers the user does not control, and heavy usage involves subscription costs. Since the NAS already stores the user's private data and is always on, it is a natural place to host a private AI service.

1.2 Project Overview

This extension installs an LLM runtime on the Raspberry Pi 4 NAS and exposes it over the local network. The main features are:

- **Local inference:** prompts and responses stay on the home or office network.
- **Shared access:** laptops, phones, and other devices can use the same model through an HTTP API or a web interface.
- **Coexistence with existing services:** the LLM runs alongside OMV, Samba, and Plex, using separate ports and a lower CPU priority.
- **Storage on the NAS drive:** model files are kept on the external drive rather than the microSD card.
- **Zero subscription cost:** all software used is free and open source.

1.3 Hardware & Software Requirements

1.3.1 Hardware

- Raspberry Pi 4 (RAM: [2 GB / 4 GB / 8 GB]; the 4 GB and 8 GB versions are recommended for LLMs)
- External USB/SATA drive already configured in OMV
- Power supply (5V, 3A)
- Heat sink and fan (important for sustained inference)
- Ethernet connection to the router

1.3.2 Software

- Raspberry Pi OS 64-bit (Debian-based, aarch64)
- OpenMediaVault and Plex Media Server (from the base project)
- Ollama: LLM runtime and model manager (uses llama.cpp internally)

- Quantized models such as Llama 3.2 (1B/3B), Qwen2.5 (0.5B/1.5B), or Gemma 2 (2B)
- PuTTY (SSH client) and `curl` for testing
- Optional: Open WebUI running on a separate PC as the chat front end

Chapter 2: Literature Survey

2.1 Existing Systems

Cloud LLM services such as ChatGPT, Gemini, and Claude offer high-quality responses and large context windows. However, they require an internet connection, charge subscription or per-token fees, and process user data on external servers.

Desktop and server LLM setups with a GPU run larger models quickly, but the hardware is expensive and power-hungry, which does not fit a low-power home NAS.

Commercial NAS vendors such as Synology and QNAP have begun to offer AI features, but these are usually tied to specific models, proprietary software, or cloud connections.

2.2 Proposed System

The proposed system runs a small quantized LLM directly on the Raspberry Pi NAS. Quantization reduces model weights from 16-bit to about 4-bit values, shrinking a model to roughly a quarter of its size so that billion-parameter models fit in the Pi's limited memory. Ollama manages model download, loading, and serving through a simple REST API.

Key benefits:

- Private: no data leaves the local network.
- Low cost: no new hardware and no subscription.
- Customizable: models can be swapped or updated at will.
- Reuses the always-on NAS, and its storage, for a new purpose.

2.3 Feasibility Study

- **Technical feasibility:** The Pi 4 has a quad-core Cortex-A72 CPU and up to 8 GB RAM. It has no GPU support for LLM inference, so inference is CPU-only. Models up to roughly 3B parameters (4-bit) are feasible on the 8 GB version; 1B-2B models suit the 4 GB version.

- **Economic feasibility:** No additional hardware is required beyond an optional fan. Software is free.
- **Operational feasibility:** Installation takes a few commands, and the model is used through a simple API or chat interface. The main operational concern is sharing CPU time with Plex.

Chapter 3: System Analysis & Design

3.1 Requirement Specifications

- **Functional:** accept text prompts over the LAN; return generated text; allow model switching; survive reboots (auto-start service).
- **Performance:** first response within a reasonable time for short prompts; no crash or out-of-memory failure while OMV and Plex are running.
- **Storage:** model files stored on the external NAS drive.
- **Security:** service reachable only from the local network; no port forwarding to the internet.
- **Resource sharing:** LLM runs at lower priority than Plex streaming.

3.2 System Flowchart

The request flow is:

1. A client (laptop, phone, or script) sends a prompt over the LAN to `http://<pi-ip>:11434`.
2. Ollama loads the model from the external drive into RAM if it is not already loaded.
3. The model generates tokens on the CPU and streams them back to the client.
4. After a period of inactivity, Ollama unloads the model and frees RAM for other services.

[Insert flowchart diagram here: Client → Router/Switch → Raspberry Pi (Ollama ↔ Model files on external drive) with OMV and Plex running alongside.]

3.3 System Design

Component	Role	Port
OpenMediaVault web UI	NAS management	80

Component	Role	Port
Samba (SMB)	File sharing	445
Plex Media Server	Media streaming	32400
Ollama	LLM API server	11434
SSH	Remote administration	22

Storage layout: a shared folder named `models` on the external drive holds all model files, so the microSD card is used only for the operating system.

Network architecture: Ollama listens on the LAN interface only. Unlike Plex, it is never port-forwarded, because Ollama has no built-in authentication.

3.4 Techniques and Algorithms Used

- **Transformer-based text generation:** the model predicts one token at a time based on all previous tokens.
- **Quantization (Q4):** weights stored as 4-bit values to reduce memory use and speed up CPU inference.
- **llama.cpp inference engine:** optimized C/C++ code that uses the ARM NEON instruction set on the Pi's CPU.
- **Memory mapping and on-demand loading:** models are loaded when requested and unloaded when idle.
- **Process priority control:** the `Nice` value lowers the LLM's CPU priority relative to Plex.

3.5 Testing Process

- **Functional testing:** the model responds correctly through the CLI, the API, and a web front end.
- **Performance testing:** measure tokens per second, time to first token, RAM use, and CPU temperature.
- **Coexistence testing:** run the LLM while streaming a Plex video and while transferring files over Samba.
- **Security testing:** confirm the service is unreachable from outside the LAN.
- **Stability testing:** run repeated prompts for an extended period and watch for throttling or crashes.

Chapter 4: Implementation Steps

4.1 Preparing the System (Week 1)

Objective: confirm the Pi can handle the added workload.

Steps:

1. Connect to the Pi using PuTTY (as in Fig 4.1.2 and 4.1.3 of the base report).
2. Update the system:

```
sudo apt update && sudo apt full-upgrade -y
```

3. Check architecture, memory, and free disk space:

```
uname -m        # should print aarch64
free -h
df -h
```

4. Check CPU temperature before starting:

```
vcgencmd measure_temp
```

Screenshot: [terminal output of `free -h` and `uname -m`]

4.2 Installing Ollama (Week 2)

Objective: install the LLM runtime and verify it runs.

```
curl -fsSL https://ollama.com/install.sh | sh
systemctl status ollama
```

The installer creates a systemd service, so Ollama starts automatically at boot.

Screenshot: [Ollama service status showing "active (running)"]

4.3 Moving Models to the NAS Drive (Week 3)

Objective: keep large model files on the external drive instead of the microSD card.

Steps:

1. In OMV, create a shared folder named `models` on the external drive.
2. Find the folder's path on the Pi (OMV mounts drives under `/srv/`):

```
ls /srv/
```

3. Give the Ollama service user ownership of the folder:

```
sudo chown -R ollama:ollama /srv/<your-drive-folder>/models
```

4. Edit the service configuration:

```
sudo systemctl edit ollama.service
```

Add the following lines, then save:

```
[Service]
Environment="OLLAMA_MODELS=/srv/<your-drive-folder>/models"
Nice=10
```

5. Restart the service:

```
sudo systemctl daemon-reload
sudo systemctl restart ollama
```

The `Nice=10` setting gives the LLM lower CPU priority than Plex, so video streams stay smooth when both are busy.

Screenshot: [OMV shared folder `models` and the `systemctl edit` screen]

4.4 Downloading and Running a Model (Week 4)

Objective: choose a model that fits the available RAM and test it.

Pi 4 RAM	Suggested models
2 GB	`qwen2.5:0.5b`
4 GB	`llama3.2:1b`, `qwen2.5:1.5b`, `gemma2:2b`
8 GB	`llama3.2:3b`, `phi3:mini`

```
ollama pull llama3.2:1b
ollama run llama3.2:1b
```

To see speed statistics for a response:

```
ollama run llama3.2:1b --verbose
```

The `--verbose` flag prints the evaluation rate in tokens per second, which is the key performance figure for the Results chapter.

Screenshot: [a sample conversation in the terminal with the eval rate shown]

4.5 Enabling Network Access (Week 5)

Objective: let other devices on the LAN use the model, while keeping it private.

Steps:

1. Add this line under `[Service]` using `sudo systemctl edit ollama.service`:

```
Environment="OLLAMA_HOST=0.0.0.0:11434"
```

2. Restart the service and test from another computer on the same network:

```
curl http://192.168.2.98:11434/api/generate -d '{
  "model": "llama3.2:1b",
  "prompt": "Explain what a NAS is in two sentences.",
  "stream": false
}'
```

3. Restrict access to the local subnet with a firewall rule:

```
sudo apt install ufw -y
sudo ufw allow from 192.168.2.0/24 to any port 11434
sudo ufw allow from 192.168.2.0/24 to any port 22
sudo ufw allow from 192.168.2.0/24 to any port 80
sudo ufw allow from 192.168.2.0/24 to any port 445
sudo ufw allow from 192.168.2.0/24 to any port 32400
sudo ufw enable
```

Check the rules carefully before enabling the firewall so that you do not lock yourself out of SSH, OMV, Samba, or Plex.

Important: Ollama has no password protection. Do not forward port 11434 on the router.

Screenshot: [curl response from a laptop; `ufw status` output]

4.6 Adding a Web Chat Interface (Week 6)

Objective: give non-technical users a simple chat page.

Open WebUI is too heavy to run on the Pi itself, so it is installed on a separate PC (for example with Docker) and pointed to the Pi's address `http://192.168.2.98:11434`. The Pi only performs inference, and the PC provides the interface.

Screenshot: [web chat interface connected to the Pi]

4.7 Testing and Validation (Weeks 7-8)

- Run the same set of prompts on each candidate model and record tokens per second.
- Stream a Plex video while the LLM is generating, and note any buffering.
- Transfer a large file over Samba while the LLM is generating.
- Monitor temperature and memory:

```
watch -n 2 vcgencmd measure_temp
htop
```

- Reboot the Pi and confirm that Ollama, OMV, Samba, and Plex all start automatically.

Screenshot: [htop showing CPU and RAM use during generation; temperature readings]

Chapter 5: Testing, Validation and Results

This chapter defines how the LLM integration is tested and records the results. Every test below is repeatable, and the benchmark script in 5.2 produces the numbers automatically, so all values in the result tables come from your own Raspberry Pi and are entered as measured.

> **Important:** The tables in this chapter list expected ranges as a guide only. Replace the empty "Measured" cells with your own results. Do not submit estimated values as measurements.

5.1 Test Environment and Methodology

Test setup

Item	Value
Device	Raspberry Pi 4 Model B, [2 / 4 / 8] GB RAM
Operating system	Raspberry Pi OS 64-bit (Debian), kernel [output of `uname -r`]
LLM runtime	Ollama [output of `ollama --version`]
Storage for models	External drive via OMV, [HDD / SSD], ext4
Network	Gigabit Ethernet, router subnet 192.168.2.0/24
Cooling	[heatsink only / heatsink + fan / none]
Background services	OMV, Samba, Plex Media Server, SSH
Ambient temperature	[ ] °C

Models under test (choose those that fit your RAM)

Model	Parameters	Quantization	Download size (approx.)
qwen2.5:0.5b	0.5B	Q4	~0.4 GB
llama3.2:1b	1.2B	Q4	~1.3 GB
gemma2:2b	2.6B	Q4	~1.6 GB
llama3.2:3b	3.2B	Q4	~2.0 GB

Procedure

1. Reboot the Pi and wait 5 minutes so temperature and memory settle.
2. Record idle temperature, free memory, and CPU load.
3. Run each model against the same four prompts (short Q&A, summarization, reasoning, writing), 3 runs per prompt.
4. Cap the output at 128 tokens so runs are comparable.
5. Record the first run separately, because it includes model loading time.
6. Repeat the key tests while a Plex stream or a Samba file transfer is active.

Metrics

Metric	Meaning	Source
Generation speed (tokens/sec)	How fast the answer is produced	`eval_count / eval_duration` from the Ollama API
Load time (s)	Time to load the model into RAM	`load_duration` from the Ollama API
Total time (s)	Request to complete answer	`total_duration` from the Ollama API
Free RAM (MB)	Memory left for other services	`/proc/meminfo`
CPU temperature (°C)	Heat during inference	`vcgencmd measure_temp`
Throttle flag	Whether the CPU was slowed by heat or low voltage	`vcgencmd get_throttled`

5.2 Automated Benchmark Script

The script below runs every prompt on every model, three times each, and saves speed, load time, temperature, free memory, and throttle status to a CSV file. Save it on the Pi as `llm_bench.py`.

```
#!/usr/bin/env python3
"""LLM benchmark for Raspberry Pi NAS (Ollama).
Usage: python3 llm_bench.py llama3.2:1b qwen2.5:0.5b
"""
import csv, json, subprocess, sys, time, urllib.request

HOST = "http://localhost:11434"
MODELS = sys.argv[1:] or ["llama3.2:1b"]
RUNS = 3
PROMPTS = {
    "short_qa": "What is a NAS? Answer in two sentences.",
    "summarize": ("Summarize in three bullet points: A NAS is a dedicated
file "
                  "storage device that lets multiple users on a network
access "
                  "shared data. It connects through Ethernet, is managed
```

```
through "
                  "a browser, and often supports RAID for redundancy."),
    "reasoning": "A drive transfers 80 MB/s. How long to copy a 4 GB file?
Show steps.",
    "writing": "Write a short email asking a teacher for a project deadline
extension.",
}

def vc(cmd):
    try:
        return subprocess.check_output(["vcgencmd", cmd]).decode().strip()
    except Exception:
        return ""

def temp_c():
    t = vc("measure_temp")          # e.g. temp=54.0'C
    return float(t.split("=")[1].split("'")[0]) if t else None

def mem_avail_mb():
    for line in open("/proc/meminfo"):
        if line.startswith("MemAvailable"):
            return int(line.split()[1]) // 1024

def generate(model, prompt):
    body = json.dumps({"model": model, "prompt": prompt, "stream": False,
                       "options": {"num_predict": 128}}).encode()
    req = urllib.request.Request(HOST + "/api/generate", data=body,
                                 headers={"Content-Type":
"application/json"})
    with urllib.request.urlopen(req, timeout=1800) as r:
        return json.load(r)

with open("llm_results.csv", "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["model", "prompt", "run", "tokens", "tokens_per_sec",
                "load_s", "total_s", "temp_before_c", "temp_after_c",
                "free_ram_mb", "throttled", "response"])
    for model in MODELS:
        for name, prompt in PROMPTS.items():
            for run in range(1, RUNS + 1):
                t0 = temp_c()
                d = generate(model, prompt)
                tps = d["eval_count"] / (d["eval_duration"] / 1e9)
```

```
                w.writerow([model, name, run, d["eval_count"], round(tps, 2),
                            round(d.get("load_duration", 0) / 1e9, 2),
                            round(d["total_duration"] / 1e9, 2),
                            t0, temp_c(), mem_avail_mb(),
                            vc("get_throttled"),
                            d["response"].replace("\n", " ")[:300]])
                f.flush()
                print(model, name, run, round(tps, 2), "tok/s")
```

Run it and keep the output file for your appendix:

```
python3 llm_bench.py llama3.2:1b qwen2.5:0.5b
column -s, -t llm_results.csv | less -S
```

Supporting commands for the other tests

```
# Disk write speed on the NAS drive (1 GB)
dd if=/dev/zero of=/srv/<drive>/speedtest.bin bs=1M count=1024 conv=fdatasync

# Disk read speed (clear cache first)
sudo sh -c 'sync; echo 3 > /proc/sys/vm/drop_caches'
dd if=/srv/<drive>/speedtest.bin of=/dev/null bs=1M
rm /srv/<drive>/speedtest.bin

# Network throughput between PC and Pi (run `iperf3 -s` on the Pi)
iperf3 -c 192.168.2.98

# Throttle status: 0x0 means no throttling or under-voltage
vcgencmd get_throttled
```

5.3 Functional Test Cases

Each case lists the steps, the expected outcome, and a result column. Mark each as Pass or Fail after running it, and attach a screenshot reference.

ID	Test case	Steps	Expected result	Result	Screenshot
F1	Service starts at boot	Reboot the Pi; run `systemctl status ollama`	Service is active (running) without manual start	[Pass/Fail]	[Fig 5.1]

ID	Test case	Steps	Expected result	Result	Screenshot
F2	Model download	`ollama pull llama3.2:1b`	Download completes; model appears in `ollama list`	[Pass/Fail]	[Fig 5.2]
F3	Models stored on NAS drive	Check `ls /srv/<drive>/models/blobs` after the pull	Model files are on the external drive, not the microSD card	[Pass/Fail]	[Fig 5.3]
F4	Local chat	`ollama run llama3.2:1b`, ask a question	A coherent answer is returned	[Pass/Fail]	[Fig 5.4]
F5	API from the Pi	`curl localhost:11434/api/generate` with a prompt	JSON reply with a `response` field	[Pass/Fail]	[Fig 5.5]
F6	API from a laptop on the LAN	Same `curl` using the Pi's IP 192.168.2.98	Same JSON reply	[Pass/Fail]	[Fig 5.6]
F7	API from a phone on Wi-Fi	Open a web chat front end pointed at the Pi	Answer is displayed on the phone	[Pass/Fail]	[Fig 5.7]
F8	Model switching	Run two different models one after another	Both respond; the first unloads when idle	[Pass/Fail]	[Fig 5.8]
F9	Idle unload	Wait 5+ minutes, then run `ollama ps`	No model listed; free RAM has recovered	[Pass/Fail]	[Fig 5.9]
F10	Reboot persistence	Reboot; check OMV, Samba, Plex, and Ollama	All four services are running	[Pass/Fail]	[Fig 5.10]
F11	Existing NAS features intact	Open an SMB share from the phone (as in Fig 4.1.5-4.1.9) and play a Plex video	File access and streaming work as before	[Pass/Fail]	[Fig 5.11]
F12	Out-of-memory handling	Request a model larger than available RAM	Request fails with an error; OMV, Samba, and Plex keep running	[Pass/Fail]	[Fig 5.12]

5.4 Performance Test Results

Copy the averages from `llm_results.csv` into the tables below. The "Expected" column shows typical community-reported ranges for a Raspberry Pi 4 with 4-bit models; your values may differ with RAM size, cooling, and drive type.

Table 5.1: Generation speed (tokens/sec, average of 3 runs, warm model)

Model	Expected	Short Q&A	Summarize	Reasoning	Writing	Average
qwen2.5:0.5b	5-10	[ ]	[ ]	[ ]	[ ]	[ ]
llama3.2:1b	3-6	[ ]	[ ]	[ ]	[ ]	[ ]
gemma2:2b	2-4	[ ]	[ ]	[ ]	[ ]	[ ]
llama3.2:3b	1-3	[ ]	[ ]	[ ]	[ ]	[ ]

Table 5.2: Load time and memory use

Model	First-run load time (s)	Total time, 128-token answer (s)	Free RAM while loaded (MB)	Free RAM at idle (MB)
qwen2.5:0.5b	[ ]	[ ]	[ ]	[ ]
llama3.2:1b	[ ]	[ ]	[ ]	[ ]
gemma2:2b	[ ]	[ ]	[ ]	[ ]
llama3.2:3b	[ ]	[ ]	[ ]	[ ]

Table 5.3: NAS baseline (unchanged by the LLM, measured with the commands in 5.2)

Test	Without LLM running	With model loaded and idle
Disk write speed, `dd` (MB/s)	[ ]	[ ]
Disk read speed, `dd` (MB/s)	[ ]	[ ]
Network throughput, `iperf3` (Mbit/s)	[ ]	[ ]
SMB large-file copy from PC (MB/s)	[ ]	[ ]

Reading the results

- Speed falls roughly in proportion to model size, since the CPU must read all model weights for every token.
- The first request is slow because the model is read from the external drive into RAM. An SSD loads noticeably faster than an HDD or the microSD card.
- A loaded model keeps its memory until the idle timeout, so free RAM for Plex and Samba drops while the LLM is active.
- A model that is idle should not change the NAS baseline in Table 5.3. If it does, check for swapping with `free -h`.

For your analysis, state which model gave the best balance of speed and quality on your Pi, and why: [your conclusion].

5.5 Coexistence, Thermal and Stability Tests

The Pi runs OMV, Samba, Plex, and the LLM on the same four CPU cores, so these tests check that the original NAS functions still work while the LLM is busy.

Table 5.4: Coexistence tests

ID	Scenario	How to run	What to record	Measured
C1	LLM alone	Run the benchmark with Plex and Samba idle	Tokens/sec (baseline)	[ ]
C2	LLM + Plex direct play	Stream a 1080p video to a TV or laptop while the benchmark runs	Tokens/sec; any buffering or stutter in the video	[ ]
C3	LLM + Plex transcoding	Force transcoding (lower quality on the client) while the benchmark runs	Tokens/sec; buffering; CPU use in `htop`	[ ]
C4	LLM + Samba transfer	Copy a 2 GB file from the PC to the NAS while the benchmark runs	Tokens/sec; file transfer speed (MB/s)	[ ]
C5	Two LLM requests at once	Send two prompts from two devices together	Time per answer; any errors	[ ]

Expected behaviour: Plex direct play and Samba copies are only lightly affected. Plex transcoding and the LLM compete for CPU, so LLM speed drops sharply and the transcoded stream may stutter unless `Nice=10` is set for Ollama.

Table 5.5: Thermal results (15-minute continuous benchmark)

Measure	No fan	With fan
Idle temperature (°C)	[ ]	[ ]
Peak temperature under load (°C)	[ ]	[ ]
Temperature after 15 minutes (°C)	[ ]	[ ]
`get_throttled` value	[ ]	[ ]
Tokens/sec at start vs end	[ ]	[ ]

A `get_throttled` result of `0x0` means no throttling occurred. Any other value means the CPU slowed down because of heat or low supply voltage, and the speed numbers in Table 5.1 may be lower than the hardware can deliver. The Raspberry Pi 4 begins to throttle at about 80 °C.

Table 5.6: Stability test (4 hours)

Check	Result
Number of prompts completed	[ ]
Failed or timed-out requests	[ ]
Out-of-memory kills (search dmesg for "killed process")	[ ]
Ollama restarts (`journalctl -u ollama`)	[ ]
Samba and Plex remained reachable throughout	[Yes/No]

5.6 Security Validation

Ollama has no built-in login, so the main security goal is that the service is reachable only from the local network.

ID	Test	How to run	Expected result	Result
S1	Listening address	`ss -tlnp` on the Pi	Port 11434 is open on the LAN address; no unexpected ports	[Pass/Fail]
S2	Firewall rules	`sudo ufw status verbose`	Port 11434 allowed only from 192.168.2.0/24	[Pass/Fail]
S3	Access from the LAN	`curl` from a laptop on the same Wi-Fi or Ethernet	Reply received	[Pass/Fail]
S4	Access from outside the LAN	Phone on mobile data: open `http://<public-ip>:11434`	Connection refused or times out	[Pass/Fail]
S5	Router port forwarding	Check the router's port-forward list	No rule forwards 11434 (only Plex 32400 if used)	[Pass/Fail]
S6	Port scan from a LAN PC	`nmap -p 22,80,445,11434,32400 192.168.2.98`	Only the expected ports are open	[Pass/Fail]
S7	OMV and SMB login still required	Open the OMV page and the SMB share without credentials	Access is denied or limited to guest rules	[Pass/Fail]
S8	Model folder permissions	`ls -l /srv/<drive>/models`	Owned by the `ollama` user; not world-writable	[Pass/Fail]

If S4 succeeds, remove the router port-forward rule at once, since anyone on the internet could use the model and the Pi's CPU.

Known limitation: the API accepts any request from an allowed LAN device. For shared or office networks, place a reverse proxy with a password or API key in front of Ollama (listed under Future Work).

5.7 Output Quality Evaluation

Speed alone does not show whether a model is useful. Read each response saved in `llm_results.csv` and score it from 1 (poor) to 5 (excellent) on three criteria:

- **Correctness:** is the information accurate?
- **Relevance:** does it answer what was asked?
- **Clarity:** is it well written and easy to follow?

Table 5.7: Quality scores (1-5)

Model	Prompt	Correctness	Relevance	Clarity	Notes
qwen2.5:0.5b	Short Q&A	[ ]	[ ]	[ ]	[ ]
qwen2.5:0.5b	Summarize	[ ]	[ ]	[ ]	[ ]
qwen2.5:0.5b	Reasoning	[ ]	[ ]	[ ]	[ ]
qwen2.5:0.5b	Writing	[ ]	[ ]	[ ]	[ ]
llama3.2:1b	Short Q&A	[ ]	[ ]	[ ]	[ ]
llama3.2:1b	Summarize	[ ]	[ ]	[ ]	[ ]
llama3.2:1b	Reasoning	[ ]	[ ]	[ ]	[ ]
llama3.2:1b	Writing	[ ]	[ ]	[ ]	[ ]
[third model]	Short Q&A	[ ]	[ ]	[ ]	[ ]
[third model]	Summarize	[ ]	[ ]	[ ]	[ ]
[third model]	Reasoning	[ ]	[ ]	[ ]	[ ]
[third model]	Writing	[ ]	[ ]	[ ]	[ ]

The reasoning prompt has a known answer: 4 GB at 80 MB/s is about 51 seconds (4096 MB / 80 MB/s = 51.2 s), so correctness can be checked directly. Small models often get arithmetic wrong, which is itself a useful finding to report.

For a fairer evaluation, have two or three classmates score the same responses without knowing which model produced them, then average the scores.

5.8 Validation Against Objectives and Summary of Results

The integration is validated by checking each objective against the evidence gathered above. Fill the last two columns once testing is complete.

Objective	Pass criterion	Evidence	Met?
Run an LLM on the Pi NAS	Model answers prompts on the Pi	F4, F5; Table 5.1	[Yes/No]
Models stored on the NAS drive	No model files on the microSD card	F3	[Yes/No]
Available to LAN devices	API and chat reachable from laptop and phone	F6, F7	[Yes/No]
Existing NAS features unaffected	SMB access and Plex playback work during LLM use	F11; C2, C4; Table 5.3	[Yes/No]
Stable operation	No crashes or out-of-memory kills in the 4-hour test	Table 5.6	[Yes/No]
Thermally safe	No throttling (`0x0`) with the chosen cooling	Table 5.5	[Yes/No]
Private and LAN-only	Not reachable from outside the network	S3, S4, S5	[Yes/No]
Usable response speed	At least [your threshold, e.g. 2] tokens/sec for the recommended model	Table 5.1	[Yes/No]

Summary of Results

- Recommended model for this Pi: [model name], at [ ] tokens/sec with [ ] MB RAM free.
- Plex direct play and Samba transfers [were / were not] noticeably affected while the LLM was generating.
- Plex transcoding together with the LLM [caused / did not cause] stutter; setting `Nice=10` [helped / made no difference].
- Peak CPU temperature was [ ] °C [with / without] a fan, and the throttle flag was [value].
- Quality: small models handled [tasks that worked] well and struggled with [tasks that failed].
- Test cases passed: [ ] of 12 functional, [ ] of 8 security.

Limitations Observed

- Speed is limited by CPU-only inference on the Pi 4, so long answers take minutes.
- Answer quality is below that of large cloud models, and answers can be wrong while sounding confident.
- The LLM and Plex transcoding share the same cores, so heavy use of both at once degrades each.
- The API has no authentication and depends on the LAN firewall for protection.

Chapter 6: Conclusion and Future Work

6.1 Conclusion

This extension showed that a low-cost Raspberry Pi NAS can also serve as a private AI assistant. By using Ollama with a small quantized model, storing models on the NAS drive, and limiting network access to the LAN, the system adds LLM capability without compromising the existing file-sharing and streaming functions. The project demonstrates that self-hosted AI is accessible even on very modest hardware, within clear performance limits.

6.2 Future Work

- **Faster hardware:** Raspberry Pi 5, or an added AI accelerator, for higher tokens per second.
- **Document Q&A over NAS files:** retrieval-augmented generation (RAG) so the model can answer questions about files stored on the NAS.
- **Authentication layer:** a reverse proxy such as Nginx with password or token protection in front of the API.
- **Separate inference server:** keep the Pi for storage and Plex, and offload the LLM to a stronger machine on the same network.
- **Automation:** use the model for photo or document tagging, and for summarizing logs and backup reports.
- **Mobile access:** a simple phone app or home-screen web page for the assistant.

Chapter 7: References

- Ollama documentation: https://ollama.com

- llama.cpp project: https://github.com/ggerganov/llama.cpp
- Open WebUI: https://github.com/open-webui/open-webui
- OpenMediaVault documentation: https://docs.openmediavault.org
- Plex Support: https://support.plex.tv
- Raspberry Pi 4 datasheet: https://www.electronicwings.com/components/raspberry-pi-4b/2/datasheet
- Base project report: "NAS Network Attached Storage & Video Streaming Using Plex", May 2025

Chapter 8: Appendices

8.1 Introduction to Large Language Models

A Large Language Model is a neural network, usually based on the transformer architecture, trained on large amounts of text to predict the next word (token) in a sequence. Repeating this prediction produces full sentences and paragraphs. Model size is measured in parameters (for example, 1B means one billion). Larger models are generally more capable but need more memory and compute.

8.2 Introduction to Quantization

Quantization stores model weights using fewer bits. A 1B-parameter model in 16-bit format needs about 2 GB; in 4-bit format it needs about 0.7 GB, plus overhead. Quality drops slightly, but the memory and speed savings are what make LLMs possible on devices like the Raspberry Pi.

8.3 Introduction to Ollama

Ollama is an open-source tool for running LLMs locally. It downloads models, manages them, and serves them through a REST API on port 11434. It supports Linux, macOS, and Windows, and uses llama.cpp as its inference engine.

8.4 Useful Commands

Task	Command
List installed models	`ollama list`

Task	Command
Show running models	`ollama ps`
Remove a model	`ollama rm <model>`
Service status	`systemctl status ollama`
View logs	`journalctl -u ollama -f`
CPU temperature	`vcgencmd measure_temp`
Memory use	`free -h`